FINSIGHT

Financial Transaction Tracker

Technical Design Document

Prepared by:

Maryam Ali

Dania Osama Bahjat

Table of Contents

Table of Contents 2
1. Introduction 4
  1.1 Purpose 4
  1.2 Scope 4
  1.3 Definitions & Acronyms 4
2. System Overview 5
  2.1 Technology Stack 5
3. Architecture 6
  3.1 High-Level Architecture 6
  3.2 Backend Layered Architecture 6
  3.3 Frontend Architecture 7
4. Key Design Decisions 9
5. Component Design 10
  5.1 Backend Components 10
    5.1.1 Controllers (REST) 10
    5.1.2 Services 10
    5.1.3 Repositories (JPA) 10
  5.2 Frontend Components 12
6. Data Design 13
  6.1 Entity Relationship Model 13
  6.2 Entity Definitions (Technical) 13
    6.2.1 users 14
    6.2.2 transactions 14
    6.2.3 fraud_alerts 14
    6.2.4 subscriptions 15
7. API Design 16
  7.1 Authentication 16
  7.2 Transaction APIs 16
  7.3 Dashboard APIs 16
  7.4 Fraud Alert APIs 16
  7.5 Subscription APIs 17
  7.6 Health APIs 17
8. Fraud Detection 18
  8.1 Scoring Rules 18
  8.2 Risk Level Classification 18

8.3 FraudDetectionResult 18
9. Subscription Detection 20
9.1 Detection Algorithm 20
9.2 Due-Soon Logic 20
9.3 Subscription Status 20
9.4 Subscription Detection Sequence Diagram 20
10. Demo Data Generation 22
10.1 Seeding Behaviour 22
10.2 Amount Ranges by Category 22
10.3 Fraud Triggers 22
11. Configuration 23
11.1 Backend 23
10.2 Docker & Compose 23
12. Testing Strategy 24
12.1 Unit Tests (JUnit 5 + Mockito) 24
12.2 Property-Based Tests (jqwik) 24
12.3 Integration Tests (Spring MockMvc) 25
12.4 Test Execution 25
13. Operational Considerations 26
13.1 Health Monitoring 26
13.2 Data Persistence Caveat 26
13.3 Performance Considerations 26
13.4 Disaster Recovery 26
16. Appendix 27
A. Diagram List 27
B. Quick-Start Commands 27
C. Known Limitations & Future Enhancements 27

1. Introduction

This Technical Design Document (TDD) provides a comprehensive technical specification for the FinSight Financial Transaction Tracker. It covers system architecture, component design, data design, API contracts, security design, deployment topology, and testing strategy. It is intended for software engineers, architects, QA engineers, and DevOps personnel responsible for building, operating, and maintaining the system.

1.1 Purpose

FinSight enables users to record, categorize, and analyse their financial transactions while automatically detecting fraudulent activity and recurring subscription payments. The system presents actionable insights through a dashboard and provides contextual alerts requiring user attention.

1.2 Scope

This document covers the Phase 1 enhancement of FinSight, which adds fraud detection, subscription tracking, demo data generation, and comprehensive filtering to the existing Spring Boot application.

Out of scope for Phase 1:

- Machine-learning-based fraud detection
- Real-time streaming transaction processing
- Multi-currency support
- External payment gateway integration
- Mobile native applications

1.3 Definitions & Acronyms

Term	Type	Definition
FinSight	Product	Financial Transaction Tracker web application
TDD	Document	Technical Design Document
JPA	Framework	Java Persistence API — ORM specification
DTO	Pattern	Data Transfer Object — decouples API schema from entity
H2	Database	In-memory relational database used in development/demo
CORS	Security	Cross-Origin Resource Sharing policy
Risk Level	Domain	LOW / MEDIUM / HIGH classification of fraud score
Fraud Score	Domain	0–100 numeric risk index computed per transaction
Subscription	Domain	Recurring payment pattern detected automatically

2. System Overview

FinSight is a full-stack web application composed of a React 18 single-page application served by nginx and a Spring Boot 3.2.0 REST API backed by an H2 in-memory database. The two services communicate exclusively over HTTP/REST using JSON payloads. Both services run as Docker containers orchestrated by Docker Compose.

Core Capabilities
Dashboard — financial overview: income, expenses, balance, fraud metrics, spending by category, etc...
Transaction Management — create, filter, paginate, and sort financial transactions with fraud score display
Fraud Detection — rule-based engine producing a 0–100 risk score with LOW / MEDIUM / HIGH classification Subscription Tracking — automatic detection of recurring payments with due-soon notifications
Demo Data — deterministic generation of 25–50 seeded transactions for new users

2.1 Technology Stack

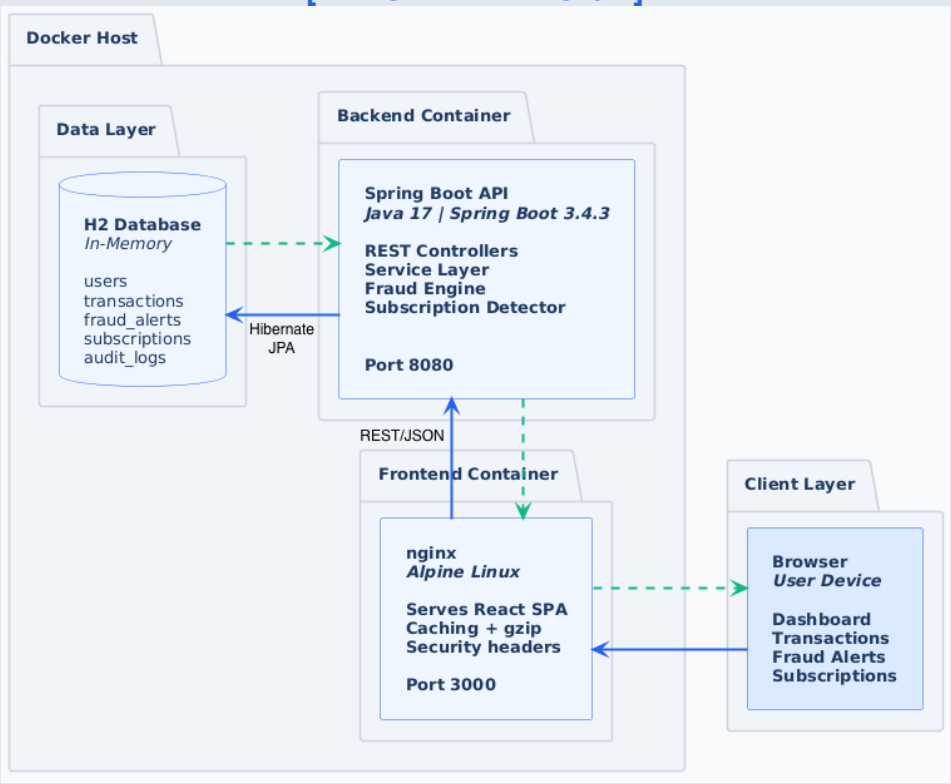
Layer	Technology / Version
Language (Backend)	Java 17
Framework (Backend)	Spring Boot 3.2.0
ORM	Spring Data JPA / Hibernate 6
Database	H2 (in-memory, dev/demo)
Build Tool	Maven 3.8+
Language (Frontend)	JavaScript (ES2022)
Framework (Frontend)	React 18 (functional components + hooks)
Router	React Router v6
Web Server (Frontend)	nginx (Alpine)
Containerisation	Docker / Docker Compose
Test Framework (Unit)	JUnit 5 / Mockito
Test Framework (Property)	jqwik

3. Architecture

3.1 High-Level Architecture

The system follows a client-server topology. The React SPA is delivered as a static bundle from nginx. All application state is managed server-side; the client communicates via RESTful HTTP requests to the Spring Boot API, which persists data to H2.

[DIAGRAM: DIAG-01]

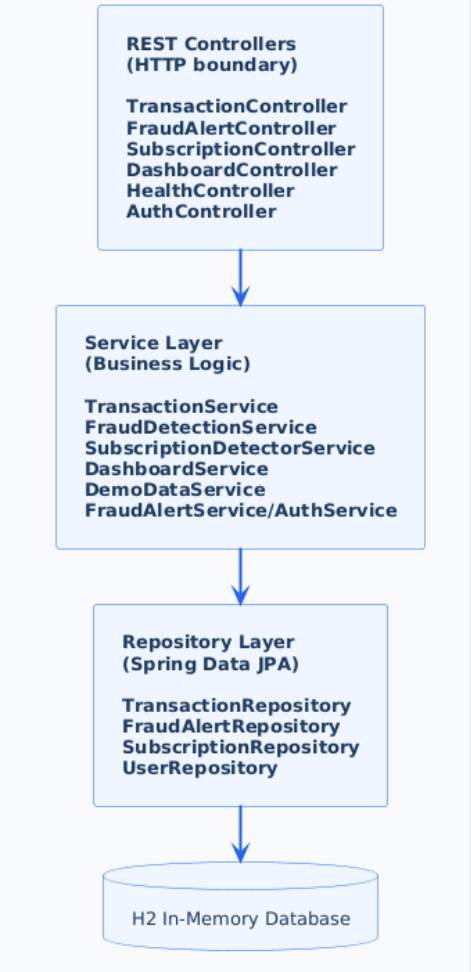


System Architecture

3.2 Backend Layered Architecture

The backend strictly follows a layered architecture pattern. Each layer has a single responsibility and communicates only with the layer immediately below it. This enforces separation of concerns, improves testability, and allows independent evolution of each layer.

[DIAGRAM: DIAG-02]



Backend Layers

3.3 Frontend Architecture

The React 18 frontend is structured into four distinct tiers: pages, components, services, and styles. Pages are route-level components that orchestrate data fetching and layout. Components are reusable, presentational building blocks. The service layer centralises all Axios HTTP calls and API endpoint definitions.

Tier	Responsibility
Pages	Route-level views: Dashboard, Transactions, FraudAlerts, Subscriptions
Components	Reusable UI primitives: Card, Button, Badge, Navbar
Services (api.js)	Axios configuration, endpoint constants, request/response handling
Styles (CSS)	Global styles (index.css) + component-scoped CSS files
React Router v6	Client-side routing; public routes (/login, /signup) redirect authenticated users to /dashboard; all other routes are protected by ProtectedRoute and redirect unauthenticated users to /login

Navigation routes:

- / → redirect to /login (unauthenticated) or /dashboard (authenticated)
- /dashboard → Dashboard.js
- /login → Login.js (public route)
- /signup → Signup.js (public route)
- /transactions → Transactions.js
- /fraud-alerts → FraudAlerts.js
- /subscriptions → Subscriptions.js

4. Key Design Decisions

Decision	Rationale
Rule-based fraud detection	Provides fully explainable, auditable results without the infrastructure complexity of ML models. Four additive rules produce a deterministic score. Chosen over ML for Phase 1 to ensure transparency and testability.
H2 in-memory database	Eliminates external database dependencies for demo deployments. Enables instant startup and reproducible test state.
Pattern-based subscription detection	Detects recurring payments with no user configuration by grouping expense transactions by category and checking 20–40 day intervals (with a 20-day deduplication filter). Two transactions with a qualifying interval constitute a detected subscription. Existing subscriptions are updated (not deleted) on re-detection, preserving IGNORED status.
Deterministic demo data generation	Seeds pseudo-random transactions using userId as the random seed. Ensures identical data across restarts for reliable demos and tests. Runs only when the user's transaction count is zero.
DTOs for API contracts	TransactionRequest / TransactionResponse and equivalent DTOs decouple the public API schema from JPA entity fields. Allows entity refactoring without breaking consumers.
JPA Specifications for filtering	Dynamic multi-predicate filtering (type, category, date range, fraud flag) is implemented via Specification<Transaction>, avoiding a proliferation of specialised repository methods.
Docker Compose orchestration	Two services (backend on port 8380, frontend on port 5733) are defined in a single docker-compose.yml with health checks and a dependency chain, ensuring the frontend waits for a healthy backend.
CORS via WebConfig	Global CORS policy (allowedOriginPatterns) is centralised in WebConfig.java rather than per-controller @CrossOrigin annotations, preventing allowCredentials conflicts.
Spring Security with JWT Authentication	JWT Bearer token authentication is implemented using Spring Security with BCrypt password hashing (via AuthService) and HS256-signed tokens (via JwtUtil). Users register and login via /api/auth endpoints; the returned token must be included in subsequent requests. DemoDataService is triggered automatically after first successful registration.

5. Component Design

5.1 Backend Components

5.1.1 Controllers (REST)

All controllers reside in finsight.controller. They handle HTTP request/response mapping, input validation, and delegation to the service layer.

Controller	Responsibilities
TransactionController	Create transaction (POST), list with filtering & pagination (GET)
FraudAlertController	List alerts with filter (GET), resolve alert (PUT /{id}/resolve)
SubscriptionController	List subscriptions from DB (GET), trigger detection (POST /detect), due-soon (GET /due-soon), ignore with user validation (PUT /{id}/ignore)
DashboardController	Aggregate financial summary with optional date range (GET)
HealthController	Liveness probe returning status and timestamp (GET)
AuthController	User registration (POST /api/auth/signup), login (POST /api/auth/login); triggers DemoDataService on first successful registration

5.1.2 Services

Services reside in finsight.service and encapsulate all business logic, fraud detection algorithms, subscription pattern analysis, and data aggregation.

Service	Responsibility
TransactionService	Orchestrates transaction creation: validates input, invokes FraudDetectionService, persists entity.
FraudDetectionService	Applies four independent scoring rules to produce a FraudDetectionResult (score 0–100, risk level, reasons list)
SubscriptionDetectorService	Groups expense transactions by category (case-insensitive), identifies recurring patterns at 20–40 day intervals with 20-day deduplication, upserts detected subscriptions preserving IGNORED status
DashboardService	Computes income/expense/balance aggregates, category breakdowns, and fraud incident counts for a given date range
DemoDataService	Seeds 25–50 deterministic transactions for a user whose count is zero; uses userId-seeded Random for reproducibility
FraudAlertService	Retrieves, filters (by resolved/severity), and resolves FraudAlert entities associated with detected fraud
AuthServiceHandles	user registration with BCrypt password hashing, credential validation on login, JWT token generation, and delegates to DemoDataService after first successful registration

5.1.3 Repositories (JPA)

All repositories extend JpaRepository and reside in finsight.repository. Custom query methods are defined using Spring Data naming conventions or @Query annotations.

Repository	Notable Custom Queries
TransactionRepository	findByUserOrderByTransactionDateDesc, countByUser, calculateAverageAmount (JPQL), findDistinctCategoriesByUser, Specification-based dynamic filter
FraudAlertRepository	findByUserOrderByCreatedAtDesc, findByUserAndResolvedAndRiskLevel (multi-filter)
SubscriptionRepository	findByUserAndStatus, findDueSoon (JPQL with date range between start and end)
UserRepository	findByUsername (login lookup)

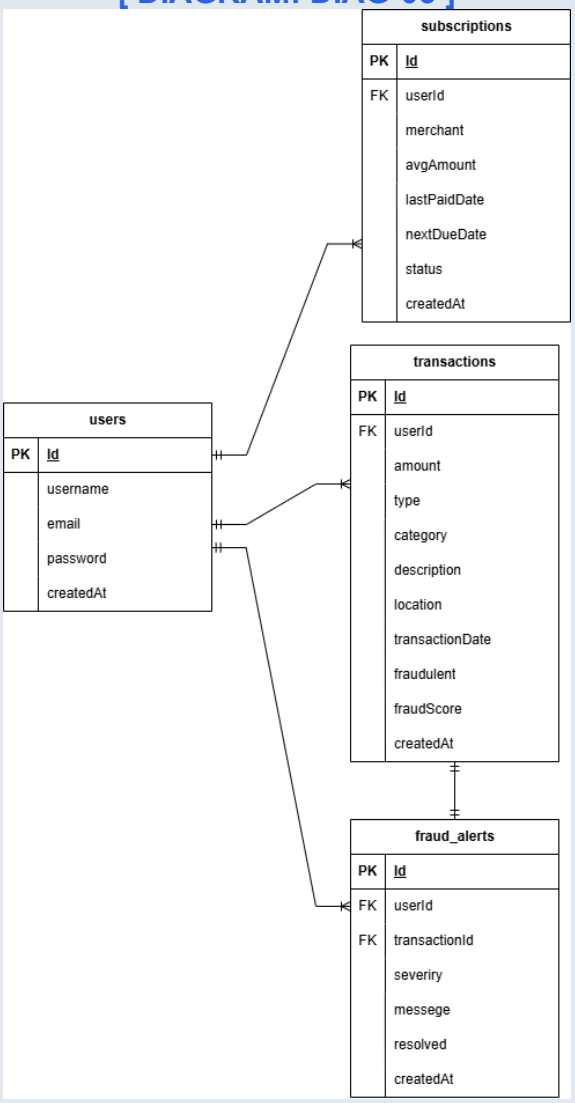
5.2 Frontend Components

Component / Page	Responsibility
Dashboard (page)	Displays income, expense, balance stat cards; spending-by-category breakdown; fraud incident count; uses GET /api/summary
Transactions (page)	Lists paginated transactions with fraud score badges; form for creating new transactions; filters: type, category, fraud status, sort direction
FraudAlerts (page)	Lists fraud alerts ordered by date desc; filter by severity/resolved; resolve alert action calling PUT /api/fraud/alerts/{id}/resolve
Subscriptions (page)	Lists auto-detected subscriptions (IGNORED excluded); highlights due-soon within 7 days with banner; ignore action with confirmation modal; manual refresh trigger; calls POST /api/subscriptions/detect and GET /api/subscriptions/due-soon on load
Navbar (component)	Top navigation bar; active route highlighting; responsive collapse on mobile
Card (component)	White container with shadow; header, body, optional footer slots
Badge (component)	Coloured pill for risk level (LOW=green, MEDIUM=amber, HIGH=red) and transaction type
Button (component)	Styled button supporting primary, secondary, danger variants; loading state
Login (page)	Public route at /login; username and password form; calls POST /api/auth/login; stores JWT token and user data in localStorage via AuthContext; redirects to /dashboard on success
Signup (page)	Public route at /signup; username, email, and password form with client-side validation (min 3-char username, min 6-char password, confirm password match); calls POST /api/auth/signup; auto-logs in on success and seeds demo data
api.js (service)	Axios instance with base URL configuration; named exports for each API call; centralised error handling

6. Data Design

6.1 Entity Relationship Model

[DIAGRAM: DIAG-03]



Entity Relationship Diagram

Cardinality summary:

- users 1 → * transactions (a user owns many transactions)
- users 1 → * fraud_alerts (a user accumulates many fraud alerts)
- users 1 → * subscriptions (subscriptions are per-user)
- transactions 1 → 0..1 fraud_alert (a fraudulent transaction generates one alert)

6.2 Entity Definitions (Technical)

6.2.1 users

Column	Type / Constraint
id	Long PK
username	String NOT NULL UNIQUE
password_hash	String NOT NULL UNIQUE
created_at	TIMESTAMP NOT NULL

6.2.2 transactions

Column	Type / Constraint
id	Long PK
user_id	Long NOT NULL FK → users(id)
amount	BigDecimal (19,2) NOT NULL
type	String(20) NOT NULL
category	String(50) NOT NULL
description	String(255)
location	String(100) nullable
transaction_date	TIMESTAMP NOT NULL
fraudulent	BOOLEAN NOT NULL DEFAULT FALSE
fraud_score	Double(5,2) nullable — 0.00 to 100.00
created_at	TIMESTAMP NOT NULL

Indexes:

- idx_transactions_user_date (user_id, transaction_date DESC) — primary lookup
- idx_transactions_fraudulent (user_id, fraudulent) — fraud alert queries
- idx_transactions_category (user_id, category) — category aggregations

6.2.3 fraud_alerts

Column	Type / Constraint
id	Long PK

Column	Type / Constraint
user_id	Long NOT NULL FK → users(id)
transaction_id	Long NOT NULL FK → transactions(id)
risk_level	String(20) NOT NULL — LOW | MEDIUM | HIGH (enum)
message	String NOT NULL
resolved	BOOLEAN NOT NULL DEFAULT FALSE
created_at	TIMESTAMP NOT NULL

6.2.4 subscriptions

Column	Type / Constraint
id	Long PK
user_id	Long NOT NULL FK → users(id)
merchant	String(100) NOT NULL — normalised name
avg_amount	BigDecimal(19,2) NOT NULL
last_paid_date	DATE NOT NULL
next_due_date	DATE NOT NULL — last_paid_date + 30 days
status	String(20) NOT NULL — ACTIVE | IGNORED
created_at	TIMESTAMP NOT NULL

Indexes:

- idx_subscriptions_user (user_id)
- idx_subscriptions_due_date (user_id, next_due_date)

7. API Design

All endpoints are prefixed /api and return application/json. Error responses use a standard ErrorResponse DTO: { "status": 404, "error": "Not Found", "message": "...", "timestamp": "..." }.

7.1 Authentication

Spring Security is configured with JWT Bearer token authentication. Users authenticate via POST /api/auth/signup (registration) or POST /api/auth/login, receiving a signed JWT token (24-hour expiry). The token must be included as a Bearer token in the Authorization header for all subsequent requests. User context (userId) is resolved from the JWT claims.

7.2 Transaction APIs

Endpoint	Details
POST /api/transactions	Body: TransactionRequest { userId, amount, type, category, description?, location?, transactionDate } Returns: TransactionResponse with fraud score and risk level Side-effects: runs fraud detection, creates FraudAlert if score ≥ 40 (MEDIUM or HIGH); marks transaction fraudulent if score ≥ 70
GET /api/transactions	Params: userId, type?, category?, startDate?, endDate?, fraudulent?, sortBy?, sortDir?, page?, size? Returns: Page<TransactionResponse> Filter via JPA Specification; default page size 20

7.3 Dashboard APIs

Endpoint	Details
GET /api/summary	Params: userId, startDate?, endDate? Returns: DashboardSummary { totalIncome, totalExpenses, currentBalance, transactionCount, fraudIncidentCount, categoryBreakdown[], timeSeriesPoints[] }

7.4 Fraud Alert APIs

Endpoint	Details
GET /api/fraud/alerts	Params: userId, resolved?, severity? Returns: List<FraudAlertDto> ordered by createdAt DESC Each DTO includes embedded TransactionResponse
PUT /api/fraud/alerts/{id}/resolve	No body required Sets resolved=true on the alert Returns: 200 FraudAlertDto | 404 if not found

7.5 Subscription APIs

Endpoint	Details
GET /api/subscriptions	Params: userId Returns: List<SubscriptionDto> fetched directly from the database (IGNORED subscriptions excluded). Does not run detection — use POST /detect for that.
POST /api/subscriptions/detect	Params: userId Runs SubscriptionDetectorService (idempotent upsert), analyses all expense transactions, creates or updates subscriptions while preserving IGNORED status, returns List<SubscriptionDto>. Called on page load and via the Refresh button.
GET /api/subscriptions/due-soon	Params: userId, days? (default 7) Returns: List<SubscriptionDto> where status=ACTIVE and nextDueDate within days from today
PUT /api/subscriptions/{id}/ignore	Sets status=IGNORED Returns: 200 SubscriptionDto | 404 if not found

7.6 Health APIs

Endpoint	Details
GET /actuator/health	Returns: { "status": "UP", "timestamp": "..." } Used by Docker health checks; requires no authentication

Key sequence steps for transaction creation:

1. Browser POSTs TransactionRequest to /api/transactions
2. TransactionController validates request via @Valid
3. TransactionService persists draft Transaction entity
4. FraudDetectionService.analyzeTransaction() computes score from four rules
5. If score ≥ 40 (MEDIUM or HIGH), TransactionService creates FraudAlert entity; if score ≥ 70, transaction is also marked fraudulent
6. Transaction entity updated with fraudScore and fraudulent flag
7. TransactionResponse (with score, risk level) returned to browser

8. Fraud Detection

The FraudDetectionService implements a rule-based additive scoring engine. Each rule is evaluated independently and contributes a fixed number of points to a cumulative score capped at 100. The engine is deterministic: identical transaction patterns always produce identical scores, enabling reliable testing.

8.1 Scoring Rules

Rule	Specification
R1 — High Amount Anomaly	+30 points Condition: transaction.amount > 3 × calculateUserAverage(user) Rationale: Unusually large transactions are a primary fraud indicator Edge case: if user has no prior transactions, average defaults to transaction amount — rule does not trigger
R2 — Rapid-Fire Activity	+25 points Condition: 5 or more transactions from the same user in any 10-minute window containing the current transaction Rationale: Automated fraud bots typically generate high-frequency transaction bursts
R3 — Geographical Anomaly	+25 points Condition: user's previous transaction occurred at a different location AND was made fewer than 2 hours prior Rationale: Physical impossibility of rapid location change indicates credential compromise
R4 — Unusual Category	+20 points Condition: transaction.category has never appeared in the user's transaction history Rationale: Fraudsters often test compromised credentials with unfamiliar merchants

8.2 Risk Level Classification

Score Range	Risk Level
0 – 39	LOW — logged, no alert created
40 – 69	MEDIUM — FraudAlert created; transaction not marked fraudulent
70 – 100	HIGH — FraudAlert created; transaction.fraudulent = true

8.3 FraudDetectionResult

The FraudDetectionService returns a FraudDetectionResult value object containing:

- double score — cumulative score (0.0 – 100.0)

- RiskLevel riskLevel — enumerated classification
- List<String> reasons — human-readable explanation for each triggered rule

9. Subscription Detection

The SubscriptionDetectorService analyses a user's transaction history to identify recurring payment patterns without any user configuration. Detection is idempotent: running detection multiple times on the same transaction set produces identical results.

9.1 Detection Algorithm

8. Retrieve all EXPENSE transactions for the user ordered by transactionDate ASC
9. Normalise category name: lowercase, trim whitespace (no suffix stripping required since categories are controlled values)
10. Group transactions by normalised category name
11. For each group with ≥ 2 transactions, apply a 20-day deduplication filter (skip any transaction within 20 days of the previous included transaction) to remove accidental duplicate payments within the same billing cycle, then compute inter-transaction intervals in days on the filtered list
12. If any 2 consecutive (deduplication-filtered) transactions have intervals in the range [20, 40] days, classify as a subscription
13. Compute avgAmount as mean of matching transaction amounts
14. Set nextDueDate = lastPaidDate + 30 days
15. Persist or update Subscription entity with status = ACTIVE

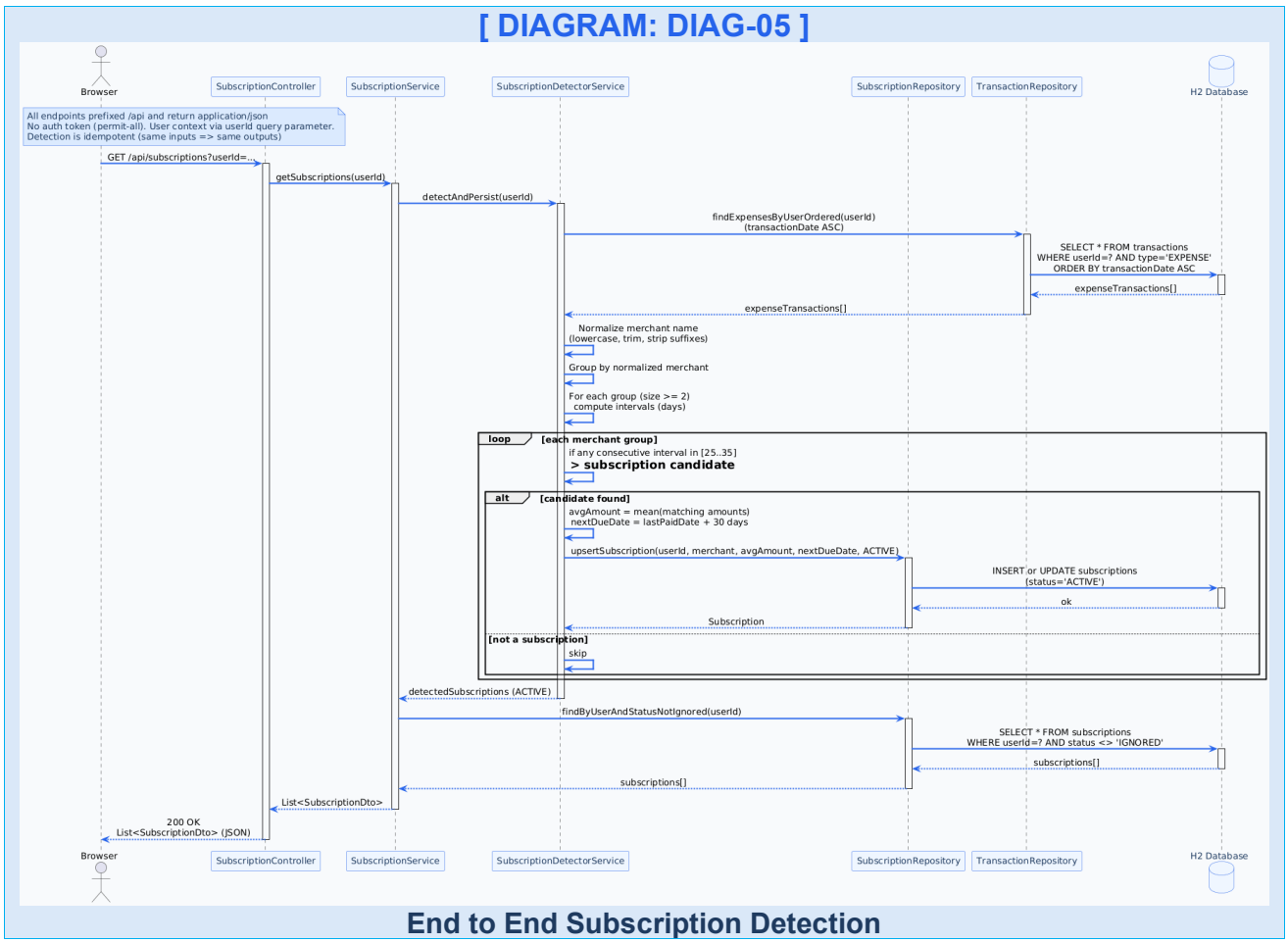
9.2 Due-Soon Logic

The GET /api/subscriptions/due-soon endpoint returns all ACTIVE subscriptions where nextDueDate falls within the next N days (default 7). The frontend displays these with a visual warning badge to prompt user attention.

9.3 Subscription Status

Status	Meaning
ACTIVE	Detected recurring payment; displayed to user
IGNORED	User dismissed; excluded from all queries and due-soon results

9.4 Subscription Detection Sequence Diagram



End to End Subscription Detection

CORS is configured globally in WebConfig.java using allowedOriginPatterns("*") with allowCredentials(true). This uses pattern matching rather than explicit origins, avoiding the Spring incompatibility that arises when combining allowedOrigins("*") with credentials. For production, allowed origins should be restricted to specific domain patterns.

10. Demo Data Generation

DemoDataService generates deterministic seed data for first-time users. The service checks the transaction count for the given userId and seeds data only when the count is zero, preventing duplicate seeding across restarts.

10.1 Seeding Behaviour

- Condition: transactionRepository.countByUser(user) == 0
- Volume: 25–50 transactions per user (determined by seeded Random)
- Date range: distributed across the past 60–90 days
- Determinism: new Random(userId) ensures identical output for the same user

10.2 Amount Ranges by Category

Category	Amount Range
Groceries	$20 – $150
Utilities	$50 – $300
Entertainment	$10 – $200
Transport	$5 – $100
Dining	$15 – $120
Healthcare	$30 – $500
Salary (INCOME)	$2,000 – $5,000
Investment (INCOME)	$100 – $2,000

10.3 Fraud Triggers

At least 3 generated transactions are crafted to trigger one or more fraud rules, ensuring the Fraud Alerts page is populated for demonstration purposes.

11. Configuration

11.1 Backend

```
# Server
server.port=8380

# H2 Database
spring.datasource.url=jdbc:h2:mem:finsight;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=false

# Actuator
management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=always
```

10.2 Docker & Compose

```
# docker-compose.yml (abridged)
services:
  backend:
    build: ./backend
    ports: ["8380:8380"]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]
      interval: 30s  timeout: 10s  retries: 3  start_period: 30s

  frontend:
    build: ./frontend
    ports: ["5733:5733"]
    depends_on:
      backend: { condition: service_healthy }
```

12. Testing Strategy

12.1 Unit Tests (JUnit 5 + Mockito)

Unit tests verify individual service methods in isolation. Dependencies (repositories, other services) are mocked with Mockito. Tests focus on specific examples and edge cases.

Test Class	Coverage
FraudDetectionServiceTest	Each rule individually (6 tests); edge cases: zero transactions, single transaction, all rules simultaneously
SubscriptionDetectorServiceTest	Pattern matching at 20-day, 30-day, 40-day intervals; category-based grouping; deduplication of payments within 20 days; non-detection when fewer than 2 qualifying transactions
DemoDataServiceTest	Seeding runs only when count=0; transaction count 25–50; date distribution 60–90 days; at least 3 fraud triggers
DashboardServiceTest	Income/expense/balance arithmetic; category breakdown aggregation; date range filtering
TransactionServiceTest	Fraud score propagated to entity; FraudAlert created when score ≥ 40; transaction marked fraudulent when score ≥ 70;

12.2 Property-Based Tests (jqwik)

Property tests verify universal correctness invariants over large randomly-generated input spaces (100 iterations per property by default). They complement unit tests by exploring edge cases that example-based tests might miss.

Property ID	Invariant
P3 — Fraud Score Threshold	transaction: score ≥ 70 → fraudulent=true; score ≥ 40 → FraudAlert created
P4 — Risk Level Mapping	score: score ∈ [0,39] → LOW; [40,69] → MEDIUM; [70,100] → HIGH
P5 — Rule Application	transaction matching rule condition: rule's points added to score
P6 — Score Bounds	inputs: 0 ≤ score ≤ 100
P18 — Subscription Pattern	daysBetween ∈ [20,40]: 2 transactions from same category detected as subscription
P19 — Due-Soon Filtering	due-soon query: all results have status=ACTIVE and nextDueDate within days
P20 — Status Update	ignore action: subscription.status = IGNORED and remains retrievable
P26 — Fraud Determinism	identical inputs: FraudDetectionService produces identical output
P27 — Subscription Idempotence	transaction set: running detection N times produces same result

Property ID	Invariant
P30 — Backward Compatibility	existing endpoint without new optional params: returns original format with HTTP 200

12.3 Integration Tests (Spring MockMvc)

Integration tests exercise full request/response cycles against a Spring application context with an embedded H2 database. They validate HTTP status codes, response body structure, and end-to-end data flow without requiring a running server.

Test Scenario	Assertions
POST /api/transactions (valid)	HTTP 201; response contains fraudScore; FraudAlert created if score ≥ 40; transaction marked fraudulent if score ≥ 70
GET /api/transactions with filters	HTTP 200; returned transactions match filter predicates; pagination metadata correct
PUT /api/fraud/alerts/{id}/resolve	HTTP 200; alert.resolved=true; 404 for unknown ID
GET /api/subscriptions	HTTP 200; idempotent; detects 1 subscription from 3 seeded matching transactions
GET /api/subscriptions/due-soon	HTTP 200; only ACTIVE subscriptions within days window
GET /api/summary with date range	HTTP 200; aggregates match manually computed totals from seeded data

12.4 Test Execution

```
# Run all backend tests
cd backend && mvn test

# Current results: 10 / 10 passing
#   6 unit tests
#   4 property-based tests (400 total iterations)
```

13. Operational Considerations

13.1 Health Monitoring

Check	Details
Backend liveness	GET /actuator/health → { status: 'UP' }; polled by Docker every 30s
Frontend liveness	wget --spider http://localhost:3000; polled by Docker every 30s
Application logs	docker-compose logs -f; Spring Boot outputs structured logs to stdout

13.2 Data Persistence Caveat

H2 is an in-memory database. All data is lost when the backend container restarts. This is intentional for Phase 1 (demo use). For production or persistent storage, replace H2 with PostgreSQL or MySQL. Environment variables for datasource URL, username, and password are externalised in application.properties and can be overridden via Docker environment settings.

13.3 Performance Considerations

Area	Design Decision
Database indexes	Three composite indexes on transactions table covering primary query patterns
Pagination	All list endpoints support page/size parameters; default size 20
Caching (planned)	Dashboard summary and subscription results can be cached with 5-minute TTL; cache invalidation on new transaction
nginx compression	gzip enabled for text/html, text/css, application/javascript, application/json
Batch inserts	DemoDataService uses saveAll() for bulk transaction seeding
N+1 prevention	@EntityGraph on FraudAlertRepository fetch to eagerly load associated Transaction

13.4 Disaster Recovery

For Phase 1, disaster recovery consists of restarting the Docker containers (docker-compose up). Demo data will be regenerated automatically. For production phases:

- PostgreSQL with regular pg_dump backups
- Docker volume mounts for data persistence
- Container image pinning to specific tags for rollback capability

16. Appendix

A. Diagram List

Diagram ID	Description
DIAG-01	System Architecture — Browser, nginx, Spring Boot API, H2
DIAG-02	Backend Layers — Controller → Service → Repository → DB
DIAG-03	Entity Relationship Diagram (ERD)
DIAG-04	End-to-End Sequence — Create Transaction with Fraud Detection
DIAG-05	Subscription Detection Flow
DIAG-06	Test Strategy Pyramid — Unit / Property / Integration / E2E

B. Quick-Start Commands

```
# Start all services (Docker — recommended)
docker-compose up --build

# Development mode
cd backend && mvn spring-boot:run
cd frontend && npm install && npm start

# Run tests
cd backend && mvn test

# Access points
Frontend:    http://localhost:3000
Backend API: http://localhost:8080/api
H2 Console:  http://localhost:8080/h2-console
Health:      http://localhost:8080/actuator/health
```

C. Known Limitations & Future Enhancements

Limitation (Phase 1)	Planned Enhancement (Phase 2+)
H2 in-memory — data lost on restart	PostgreSQL with Docker volume persistence
JWT authentication implemented (Phase 1)	Role-based access control (RBAC) and OAuth 2.0 / SSO integration
Rule-based fraud only	ML model (scikit-learn) via REST microservice
No real-time notifications	WebSocket push for instant fraud alerts
Single-node deployment	Kubernetes with horizontal pod autoscaling
No export functionality	CSV/PDF transaction export
Single currency (USD)	Multi-currency with exchange rate API
No user preferences	Customisable fraud thresholds and notification settings